Fondements de la programmation parallèle

Table des matières

1 Introduction **1**
1.1 Modèles, environnements, paradigmes de programmation 1

2 Les sources du parallélisme **1**
2.1 Parallélisme de données 2
2.2 Parallélisme de tâches (flots) 2
2.3 Parallélisme de flux 2

3 Exemples de parallélisation **2**
3.1 Parallélisme de données 2
3.2 Parallélisme de tâches 2
3.3 Parallélisme de flux 3

4 Granularité et degré de parallélisme **4**
4.1 Degré de granularité (degré de parallélisme) 4
4.2 Degré de parallélisme 4
4.3 Exercice 1 4

5 Choisir T_{opt} sans connaître le nombre de processeurs **5**

6 Accélération et efficacité **6**

1 Introduction

1.1 Modèles, environnements, paradigmes de programmation

Formes de parallélisme.

Parallélisme implicite

Le programmeur ne s'occupe pas d'écrire le code parallèle.

— Une couche supplémentaire est ajoutée au compilateur (autre que l'analyse syntaxique et grammaticale...).

— **Inconvénient** : un problème à plein potentiel de parallélisme peut ne pas être détecté par un paralléliseur automatique.

Exemple

Code séquentiel :

$$\begin{array}{ll} S_1 : & x := f() \\ S_2 : & y := x + \cdots \\ & \vdots \\ S_3 : & x \mathrel{+}= g() \\ S_4 : & z := \cdots + x \end{array}$$

Le compilateur voit : $S_1 \to S_2 \to S_3 \to S_4$ (chaîne séquentielle), alors qu'en réalité S_1, S_2 sont indépendants de S_3, S_4.

C'est pourquoi il faut que le programmeur ait une certaine connaissance des possibilités de parallélisme du compilateur. On réécrit par exemple :

$$\begin{array}{lll} S_1, S_2 & \text{(inchangés)} & \\ S_3 : & t := g() & \Longrightarrow \\ S_4 : & z := \cdots + t & \end{array} \qquad \begin{array}{ccc} S_1 & & S_3 \\ \downarrow & \| & \downarrow \\ S_2 & & S_4 \end{array}$$

Parallélisme explicite

L'utilisateur est celui qui écrit le code parallèle. Cela donne plus de contrôle et une meilleure résolution.

2 Les sources du parallélisme

— Exploiter toute la puissance de calcul d'une machine est difficile.

— **Source** : éléments qui donnent des indications sur la possibilité de parallélisme.

2.1 Parallélisme de données

Un même traitement sur plusieurs données : pas de dépendances entre les données.

Exemples : changer la luminosité d'une image ; changer chaque « Bnj » en « Bonsoir » dans un fichier.

→ On aura une dépendance quand on va les diviser au début : partage de données en mémoire distribuée (DM) (communication).

On exploite la régularité des données : matrice, vecteur, liste homogène ⇒ données régulières.

2.2 Parallélisme de tâches (flots)

Consiste à appliquer des calculs différents sur des données différentes afin de générer du parallélisme.

— C'est fait à travers une étude et de plusieurs manières (étude des dépendances) ; on choisit la meilleure en termes de temps et de processeurs : **compromis ressources–temps**.

2.3 Parallélisme de flux

Technique de travail à la chaîne.

— Chaque donnée subit une séquence de traitements réalisée en mode *pipeline*, en exploitant une régularité des données.

— On utilise des machines pipelines (CPU destiné à une tâche).

Lorsqu'on combine plus d'un mode, on parle de **parallélisme mixte**.
Exemple : plusieurs tâches en parallèle dont une tâche avec des données parallèles.

3 Exemples de parallélisation

3.1 Parallélisme de données

pour i de 1 à n **faire**
$A[i] \leftarrow A[i] + 4$
fin pour

L'idéal ici est d'avoir n cœurs :

$$\left.\begin{array}{l} T_1 = n \\ T_n = 1 \end{array}\right\} \quad S = \frac{n}{1} = n, \qquad E = \frac{n}{n} = 1.$$

3.2 Parallélisme de tâches

$$\begin{array}{ll} t_1 : & a \leftarrow a + 2 \\ t_2 : & b \leftarrow b - 4 \\ t_3 : & d \leftarrow d + 9 \\ t_4 : & c \leftarrow a * (b - d) \end{array}$$

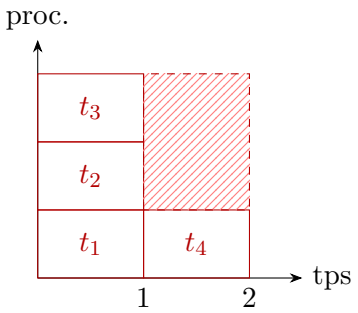
Version V_1 : $t_1 \parallel t_2 \parallel t_3 \rightarrow t_4$.
Pas de dépendances (lecture/écriture ou écriture/écriture sur des variables communes) entre t_1, t_2, t_3.
On exécute en 2 ut au lieu de 4 ut :

$$T_1 = 4, \quad T_3 = 2 \text{ (3 processeurs)} \quad \Rightarrow \quad S = \frac{T_1}{T_3} = 2.$$

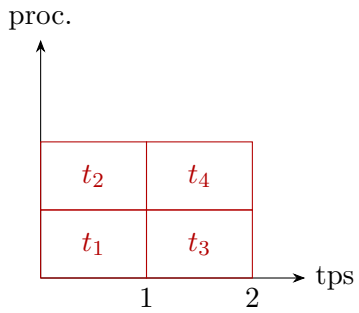
Si on change $t_4 : c \leftarrow a * b$:

— V_1 reste possible ;
— on peut aussi faire $(t_1 \parallel t_2) \rightarrow (t_3 \parallel t_4)$;
— le speedup = 2 ne change pas.

$$\text{efficacité} = \frac{\text{speedup}}{\text{nb de processeurs}} \quad \Rightarrow \quad E_3 = \frac{2}{3} \approx 0{,}666 \ (66\%), \qquad E_2 = \frac{2}{2} = 1 \ (100\%).$$



6 places, 4 remplies $\Rightarrow E = \frac{4}{6} = \frac{2}{3}$



$E = \frac{4}{4} = 1$

3.3 Parallélisme de flux

pour i de 1 à n **faire**
$C[i] \leftarrow ((A(i) \underset{O_1}{+} 2) \underset{O_2}{*} B(i)) \underset{O_3}{/} 3$
fin pour

→ On affecte chaque opération O_j au processeur P_j.

Notation : $O_j(i)$ où i est l'indice de l'itération.

t	$P_1 : O_1$	$P_2 : O_2$	$P_3 : O_3$	
1	$O_1(1)$	–	–	latence
2	$O_1(2)$	$O_2(1)$	–	
3	$O_1(3)$	$O_2(2)$	$O_3(1)$	$\leftarrow C(1)$
4	$O_1(4)$	$O_2(3)$	$O_3(2)$	dès $t = 4$ tous les processeurs travaillent
⋮	⋮	⋮	⋮	
n	$O_1(n)$	$O_2(n-1)$	$O_3(n-2)$	
$n+1$	–	$O_2(n)$	$O_3(n-1)$	
$n+2$	–	–	$O_3(n)$	$\leftarrow C(n)$

Avec $p = 3$:

$$T_1 = 3n, \qquad T_3 = n + 2 = n + \underbrace{p - 1}_{\text{latence}}, \qquad S_3 = \frac{3n}{n+2} \simeq 3, \qquad E_3 = \frac{n}{n+2} \simeq 1.$$

Généralisation

$$T_1 = pn, \qquad T_p = n+p-1, \qquad S_p = \frac{pn}{n+p-1} \simeq p, \qquad E_p = \frac{n}{n+p-1} \simeq 1 \qquad (\text{si } p \lll n).$$

4 Granularité et degré de parallélisme

4.1 Degré de granularité (degré de parallélisme)

Grain : taille moyenne des tâches élémentaires qui ont guidé la parallélisation.

— Si communication lente ⇒ granularité **grosse**.

— Si communication rapide ⇒ granularité **fine**.

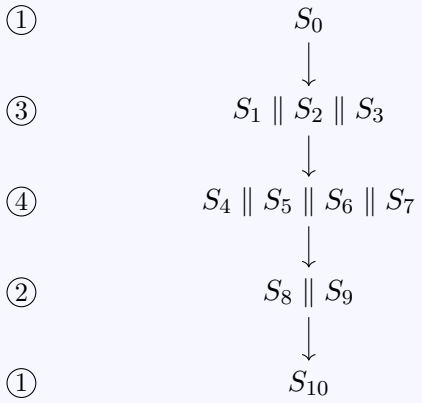
→ La granularité est liée à l'architecture : nombre de processeurs / communication (mémoire partagée SM, mémoire distribuée DM).

4.2 Degré de parallélisme

Mesure le nombre de sous-tâches exécutées simultanément.

Exemple

Graphe de tâches organisé par niveaux (le nombre de tâches parallèles par niveau est entre parenthèses) :



— $D_{\max} = 4$: plus grand nombre de tâches indépendantes pouvant être exécutées simultanément à un même niveau.

— $D_{\min} = 1$.

— $D_{\text{moy}} = \dfrac{1+3+4+2+1}{5} = \dfrac{11}{5}$.

Remarques.

— $D_{\max}$ donne le meilleur temps d'exécution T_{opt} (temps optimal parallèle).

— Mais pas toujours le nombre de processeurs optimal P_{opt}.

— On peut avoir $D_{\max}$ = nb de processeurs, mais pas forcément = P_{opt} (P_{opt} = nombre minimal de processeurs pour avoir T_{opt}).

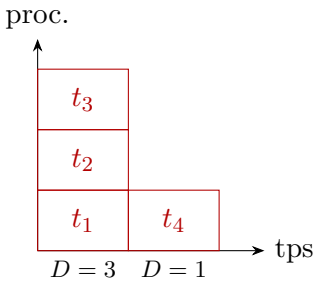
— T_{opt} est indépendant du nombre de processeurs.

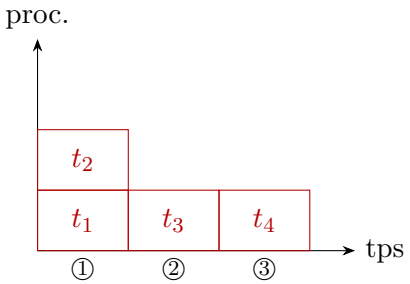
4.3 Exercice 1

$$\begin{array}{ll} t_1 : & a \leftarrow a+2 \\ t_2 : & b \leftarrow c-4 \\ t_3 : & d \leftarrow d+9 \\ t_4 : & c \leftarrow a*b-d \end{array} \qquad \begin{array}{c} t_1 \parallel t_2 \parallel t_3 \\ \hline t_4 \end{array} \downarrow \text{temps}$$

— $D_{\max} = 3, \quad D_{\min} = 1, \quad D_{\text{moy}} = \dfrac{1+3}{2} = 2.$

— $T_{\text{opt}} = 2, \quad P_{\text{opt}} = 3.$



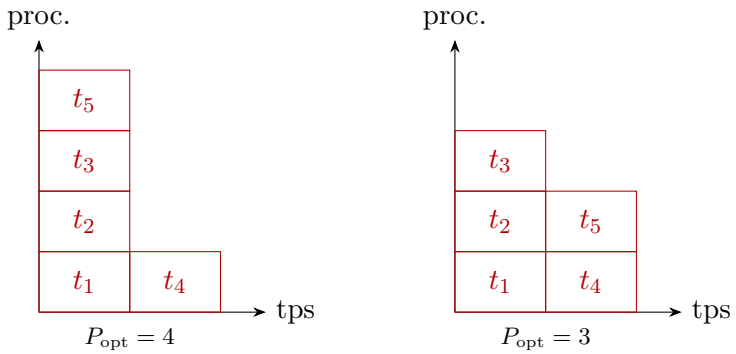


Dans ce cas : $T_{\text{opt}} = P_{\text{opt}}$.

Si on ajoute $t_5 : e \leftarrow e + 1$:

$$\frac{t_1 \parallel t_2 \parallel t_3 \parallel t_5}{t_4}$$

— $D_{\max} = 4, \quad D_{\min} = 1, \quad D_{\text{moy}} = \frac{5}{2}, \quad T_{\text{opt}} = 2.$



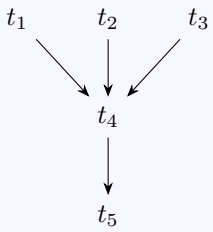
On peut faire les tâches avec un nombre de processeurs inférieur et avec le même temps :

$$P_{\text{opt}} < D_{\max}.$$

5 Choisir T_{opt} sans connaître le nombre de processeurs

On utilise le **graphe de dépendance / précédence**.

Exemple 1

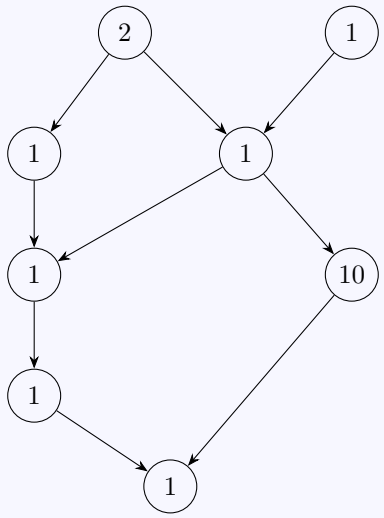


$T_{\text{opt}} = 3$

$T_{\text{opt}} = 3$: longueur de la chaîne la plus longue, comptée en sommets ($t_1 \to t_4 \to t_5$).

Exemple 2

Ici on a une *dépendance redondante* : le graphe n'est pas un arbre.



— Chaîne la plus longue = 5 si aucune taille de tâche n'est indiquée.

— La plus longue en tenant compte de la taille des tâches : $2 + 1 + 10 + 1 = 14$.

$$T_{\text{opt}} = \sum \text{longueurs des tâches constituant un chemin (le plus long)}$$

6 Accélération et efficacité

Évaluation des performances des programmes parallèles : **accélération** et **efficacité**.
On doit évaluer pour savoir l'efficacité.

— $T_1(n)$: temps d'exécution séquentiel (1 processeur).

— $T_p(n)$: temps d'exécution parallèle (p processeurs).

$$\text{Accélération (speedup)} \;=\; S_p(n) = \frac{T_1(n)}{T_p(n)}, \qquad \text{Efficacité} \;=\; E_p = \frac{S_p}{p}.$$

— Si ≈ 1 : aucun gain $\rightarrow$ le séquentiel est mieux.

— Si $= p$: c'est le cas idéal, bonne communication (pas de ralentissement).

— Si pas de dépendances et même durée : $A = \dfrac{\text{nb de tâches}}{\text{nb de processeurs}}$.

— Si $> p$ (**supralinéaire**) : cas surprenant qui arrive à cause de la mémoire.

  — En séquentiel, le processeur unique n'a pas assez de mémoire pour toutes les données $\rightarrow$ il utilise le disque $SWAP$ (lent) :

$$T_1 = \text{tps de swap} + \text{tps de traitement}.$$

  — En parallèle, les données sont réparties, chaque processeur garde sa part en mémoire (pas de swap, ou très peu) :

$$T_p = \frac{\text{tps de traitement}}{p}.$$